

Designing a Generative AI Developer Assistant to Accelerate Front-End Delivery in a Regulated Airline Environment

Programme: UCD Professional Academy - Advanced Artificial Intelligence

Author: Eve Reilly

Submission Date: 30/04/2025

Table of Contents

Table of Contents **2**
Executive Summary 3
1. The Developer Inefficiency Problem in Airline Software Teams 4
2. Rise of Generative AI in Software Engineering 5
3. Designing the AI Assistant Workflow 6
a) Ticket Interpretation Prompt 7
b) UI Scaffold Generation 7
c) Test Case Generation 7
d) Documentation Generation 7
e) Pull Request Description 7
Workflow Validation 8
4. Hallucinations, Trust, and Human Oversight in AI-Generated Code 8
5. Data Protection, GDPR, and Airline Compliance Risks 9
6. Leadership and Introducing AI to Development Teams 10
7. Measured Results from Practical Tests 11
8. Limitations and Future Potential 13
9. Reflection on the AI Journey 14
Conclusion 15
Bibliography 16
Appendix A: AI Usage Guideline for Front-End Developers 18

Executive Summary

This study examines the design, implementation, and evaluation of a Generative AI-assisted workflow aimed at improving front-end development efficiency within a highly regulated airline software environment. Instead of treating large language models as automated coding engines, the work presents them as cognitive assistants that help with the thinking, planning, and structuring phases of development. The study explores how these models can support developers in interpreting ambiguous requirements, organising tasks, creating documentation, and speeding up delivery without compromising safety, accuracy, or compliance.

A central challenge identified in airline software teams is that much of a developer's time is spent not on writing code but on interpreting unclear Jira tickets, navigating extensive documentation, and producing supporting artefacts such as test cases, technical summaries, and pull request descriptions. These activities create delays, increase cognitive load, and lead to inconsistencies that can spread through the development lifecycle. Addressing these inefficiencies presents a genuine opportunity to improve both the developer experience and overall delivery performance.

To investigate this, a structured AI workflow was built and tested using real front-end development tasks, including the creation of a new "Seats" component. The workflow covered requirement interpretation, UI scaffolding, test case generation, documentation drafting, and pull request preparation. Across these stages, the AI demonstrated notable productivity gains, with average time savings of about 70% for pre- and post-development activities. Developers also reported less cognitive strain and greater clarity when beginning tasks. Importantly, the evaluation showed that the benefits extended beyond individual productivity: the increased consistency of outputs improved cross-team communication and reduced the friction often associated with onboarding or handovers.

However, the study also highlights key risks, such as hallucinations, regulatory limits under GDPR, and the danger of over-reliance on AI-generated outputs. These findings emphasise that successful adoption depends on strong governance, human-in-the-loop validation, and leadership-driven change management. Leaders play a vital role in setting expectations, addressing concerns about job displacement, and ensuring that AI is seen as an augmentation tool rather than a replacement for developer expertise. Clear communication, structured training, and visible sponsorship are essential for building trust and embedding AI safely into existing workflows. Overall, the study concludes that Generative AI can deliver significant efficiency benefits when applied responsibly, with clear boundaries, solid oversight, and thoughtful leadership guiding its integration into everyday development practice.

1. The Developer Inefficiency Problem in Airline Software Teams

Modern airline organisations operate within complex digital ecosystems that include booking engines, online check-in flows, loyalty programme platforms, ancillary revenue systems, and companion mobile applications. Each system connects seamlessly with backend services, third-party providers, regulatory requirements, and brand guidelines. Because of this, front-end development teams work at the intersection of many stakeholders, including product owners, UX designers, backend engineers, accessibility specialists, legal and compliance teams, and operational staff.

In this environment, much of a front-end developer's time is not actually spent writing code but instead interpreting unclear Jira tickets, navigating Confluence documentation, deciphering API specifications, and ensuring compliance with accessibility and brand standards. Beyond implementation, developers also need to write test cases, prepare technical documentation, draft pull request descriptions, and estimate tasks even when information is incomplete or shifting. These activities often create friction, delays, miscommunication, context switching, and a heavy cognitive load that slows progress before development has even properly started.

Traditional tools like ticketing systems such as Jira and IDEs like Visual Studio Code tend to be passive, storing information without helping developers interpret, prioritise, or organise it into actionable steps. Developers must manually turn scattered information into a clear plan of action. This gap offers a strong opportunity for Generative AI to act as a cognitive assistant that supports developer understanding, organises unrefined inputs, and accelerates task initiation in a practical way.

These inefficiencies have real business consequences. Delays in interpreting requirements slow the release of customer-facing features, extend sprint cycles, and increase the risk of defects reaching production due to misunderstandings formed early in the process. In an airline context where digital channels directly influence customer satisfaction, operational efficiency, and ancillary revenue, even small delays in front-end delivery can have a disproportionate effect. Improving how developers interpret and act on information is therefore not only a question of productivity but also a strategic opportunity to strengthen digital delivery across the organisation.

2. Rise of Generative AI in Software Engineering

Recent advances in Generative AI, particularly large language models such as OpenAI's ChatGPT and AI-assisted development tools like GitHub Copilot, have significantly changed how software developers work with code, documentation, and technical requirements. Instead of relying on predefined commands, these systems respond to natural-language prompts, perform a kind of on-the-fly reasoning, and produce human-like structured outputs that support many development tasks (Brown et al., 202). Bommasani et al. (2021) present the idea of "foundation models", large general-purpose systems that can be adapted to numerous downstream tasks, including those in software development.

Unlike earlier development tools that required precise inputs, Generative AI can interpret ambiguous requirements and turn them into actionable development guidance. It can shorten lengthy documentation into clear development plans, create code scaffolds and test cases, summarise complex API interactions, and generate technical documentation, allowing developers to engage with information conversationally instead of manually searching across multiple platforms.

This shift represents a move from tool-based development to AI-augmented cognition, where AI systems act as intelligent assistants that support reasoning, planning, and decision-making rather than replacing human expertise (Russell and Norvig, 2021). The developer's role moves from information processor to decision-maker and reviewer of AI-assisted outputs. McKinsey (2023) estimates that generative AI could reduce time spent on documentation and test creation by up to 45%, allowing developers to redirect effort toward higher-value problem-solving work.

For airline businesses, where digital systems are both operationally critical and tightly regulated, this capability brings major business implications. Faster interpretation of requirements speeds up the delivery of customer-facing features across booking, check-in, and self-service platforms. Lower cognitive load can help ease developer burnout in high-pressure environments. Greater consistency in documentation and testing improves software quality and reduces the risk of production defects reaching customers.

3. Designing the AI Assistant Workflow

A repeatable Generative AI workflow was created to support front-end development tasks within a structured and regulated airline environment. Instead of using AI in an ad-hoc way, the goal was to define consistent prompt templates that could be applied across the entire development lifecycle, from interpreting initial requirements to final delivery and documentation.

To test the workflow in practice, a representative use case was chosen: building a new “Seats” component within a customer-facing booking interface. It required a header, sub-header, image, and a call-to-action button linking to a dedicated seat selection page. This kind of feature is common in airline front-end development, combining UI composition, navigation logic, and user journey design. The five-stage workflow is shown in Figure 1 below.

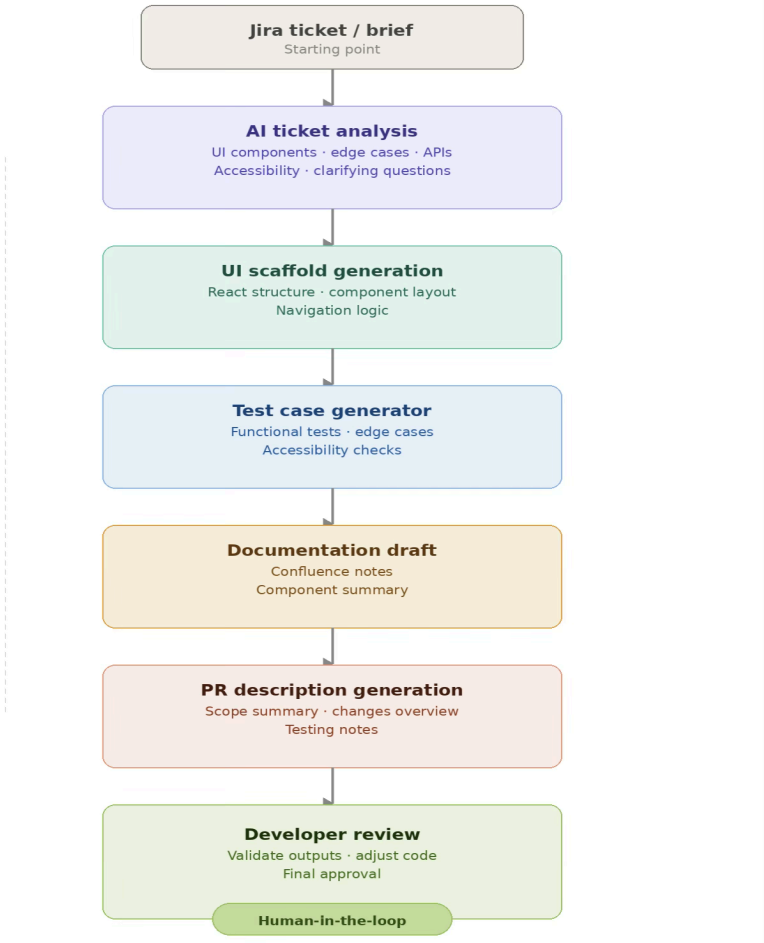


Figure 1: Generative AI-assisted front-end development workflow

a) Ticket Interpretation Prompt

The first stage used Generative AI to interpret the initial feature description as if it were a Jira ticket. The AI was prompted to break down the requirement into structured outputs, including required UI components, expected user interaction flows, potential edge cases, accessibility factors, possible API dependencies, and clarifying questions for product or UX stakeholders. This structured breakdown improved the clarity of the requirement before any coding began, revealing ambiguities that might otherwise have appeared mid-development.

b) UI Scaffold Generation

Using the structured output from the previous stage, the AI was prompted to generate a basic front-end scaffold for the Seats component. This included an Angular based component structure with placeholder elements for the header, sub-header, image, and navigation button. The aim was not to produce production-ready code but to speed up initial setup, reduce blank-canvas friction, and ensure structural alignment with the interpreted requirements before detailed implementation started.

c) Test Case Generation

The AI was then used to generate both functional and edge case test scenarios. These covered verifying correct rendering of UI elements, ensuring navigation worked as expected when the button was clicked, and validating accessibility requirements such as screen reader compatibility and keyboard navigation. Generating these scenarios from the structured requirement output meant testing considerations were identified earlier in the process than usual.

d) Documentation Generation

Once the component was conceptually defined and implemented, the AI produced a structured summary suitable for internal documentation systems like Confluence. This included a description of the component, its role in the user journey, and key implementation details. Having a structured draft to refine turned out to be much faster than writing documentation from scratch after the development work was finished.

e) Pull Request Description

Finally, the AI generated a structured pull request description outlining the scope of changes, implementation summary, and suggested testing steps. This helped maintain consistency in communication among reviewers and reduced time spent manually drafting PR documentation. Reviewers noted that the standardised format made it easier to understand the intent and scope of changes without needing to inspect the code directly.

Workflow Validation

This workflow was tested across several development tasks, including the Seats component example, to measure its effectiveness, reliability, and practicality in a real-world front-end development setting. Each stage was assessed for both time efficiency and the quality and usefulness of the AI-generated output. The detailed results of these tests are presented in Section 7.

4. Hallucinations, Trust, and Human Oversight in AI-Generated Code

A critical learning was that Generative AI cannot be trusted blindly in a software engineering context, especially within a regulated airline environment where accuracy, traceability, and compliance are essential. While the AI was highly effective at interpreting and structuring information, it also showed a consistent tendency to produce confident but incorrect outputs when left unconstrained, a behaviour widely referred to as “hallucination” (Bender et al., 2021). These hallucinations did not appear as obvious mistakes; instead, they often looked plausible and well-structured, making them easy to mistake for correct technical guidance if not carefully reviewed.

Several recurring issues appeared during testing. The AI often made wrong assumptions about API response structures, invented properties or helper functions that did not exist, and at times misinterpreted business logic described in ticket requirements. In some cases, accessibility handling was reduced to generic advice that did not fully match the real-world compliance standards required in airline applications. What made this more concerning was that these issues were not always visible during a quick review, which reinforced the need for systematic and methodical validation of every AI-generated output before using it in development.

These findings showed that AI-generated outputs should never be treated as authoritative. A human-in-the-loop approach was therefore adopted, where all AI responses were treated as draft artefacts requiring developer validation, correction, and contextual adaptation. This approach aligns closely with the European Commission’s (2021) AI Act framework, which emphasises the importance of meaningful human oversight in higher-risk AI use cases. Prompts were refined step by step to restrict the AI strictly to the information provided, require it to state its assumptions, and highlight areas of uncertainty. The idea of “trust calibration” is the understanding of what the AI can and cannot reliably do, which has become an essential practical skill for developers (Amershi et al., 2019).

By combining careful prompt design with steady human oversight, the reliability and usefulness of AI outputs improved noticeably. The shift in mindset from seeing AI as an authority to recognising it as a capable yet fallible collaborator became one of the most important practical lessons learned.

5. Data Protection, GDPR, and Airline Compliance Risks

Airlines operate in one the most tightly regulated digital environments of any industry, especially when it comes to passenger data, payment information, and operational systems. The introduction of Generative AI into the development workflow must therefore be examined through the lens of governance and data protection from the very beginning, following the guidance of the Irish Data Protection Commission (DPC, 2023). This creates a higher risk profile when using external AI tools, since even small code fragments, screenshots, or feature descriptions might include or indirectly refer to regulated data categories.

Several key governance factors influenced the approach taken in this study. No real passenger data, booking records, or operational information were entered into any AI system at any stage. All prompt examples were anonymised, fictionalised, or rewritten in abstract terms to ensure that no personally identifiable information was shared with external platforms, consistent with GDPR requirements (European Union, 2016). Proprietary source code, internal API structures, and architectural details were also intentionally left out of prompts, lowering the risk of revealing intellectual property or sensitive implementation knowledge to third-party services.

The use of cloud-based AI tools also brings up significant questions about data residency, retention, and processing outside the airline's controlled infrastructure. The Irish Data Protection Commission, in line with EU GDPR guidance, states that organisations must carry out Data Protection Impact Assessments (DPIAs) whenever personal data could be involved in AI-supported processes. This highlighted the importance of having well-defined internal policies that govern acceptable AI use by developers and confirmed that AI adoption should be integrated into existing compliance frameworks rather than treated as a separate technical trial.

As a result, a draft AI Usage Guideline for front-end developers was created and is included in Appendix A. The guideline sets out approved uses, restricted data categories, prompting best practices, and required validation steps for AI-generated outputs. These measures show that broader adoption of AI-assisted development within an airline environment demands not only technical capability but also formal governance, structured training, and continuous oversight from leadership, security, and compliance teams.

6. Leadership and Introducing AI to Development Teams

The successful adoption of Generative AI within development teams is not primarily a technical challenge but a leadership and change-management one. Kotter's (1996) eight-step model for leading change is particularly relevant: without a clear sense of urgency, a visible leadership coalition, and early demonstrable wins, new technologies often remain confined to isolated experimentation rather than becoming embedded in standard practice. Simply providing access to AI tools is not sufficient; teams require direction, reassurance, and structured support to use them effectively and responsibly.

One of the first barriers leaders must address is the perception that AI may replace developer roles. Research by Accenture (2023) found that 64% of workers in skilled technical positions expressed concern about AI-driven job displacement. In practice, the findings here show that Generative AI functions most effectively as an augmentation tool that supports thinking, planning, and documentation rather than replacing core engineering expertise. Communicating this distinction clearly and early is vital to prevent resistance, anxiety, and disengagement among team members.

Leaders must also confront the widespread lack of understanding regarding AI's limitations. Developers may overestimate the reliability of AI-generated outputs and place too much trust in them, or conversely underestimate their value due to poor early experiences. Training in prompt design, critical evaluation of AI responses, and human-in-the-loop validation therefore becomes essential. Organisations that invest in structured AI literacy programmes consistently achieve higher and safer adoption rates than those relying on informal, self-directed experimentation (Daugherty and Wilson, 2018).

Finally, AI tools should be thoughtfully integrated into existing workflows rather than introduced as optional extras. Embedding AI support into everyday activities such as ticket analysis, test case creation, documentation, and pull request preparation allows developers to experience tangible benefits without disrupting established routines. The World Economic Forum (2023) notes that AI adoption is most successful when it reduces friction in current work instead of requiring a complete process redesign.

7. Measured Results from Practical Tests

To evaluate how effective the AI-assisted workflow was, the process was applied to five representative front-end development tasks over two sprint cycles, including the “Seats” component from section 3. Time spent on key pre- and post-development activities was recorded both with and without AI assistance, allowing a direct, like-for-like comparison. This approach ensured the assessment relied on measurable outcomes rather than subjective impressions. The collected data showed clear and repeatable efficiency gains across all workflow stages.

The table below summarises average time measurements across the five tasks for each workflow stage:

Workflow Stage	Without AI (avg.)	With AI (avg.)	Time Saving	Key Observation
Ticket interpretation & planning	45–60 min	12–18 min	~65%	Edge cases and clarifying questions surfaced quickly
UI component scaffold setup	30–45 min	10–15 min	~65%	Eliminated blank-canvas friction; required adaptation
Test case generation	30–40 min	8–12 min	~70%	Accessibility scenarios identified earlier and more comprehensively
Technical documentation	25–35 min	6–10 min	~73%	AI drafts structurally complete; minor editing required
Pull request description	15–20 min	3–6 min	~75%	Consistent format improved reviewer comprehension
Total per feature (approx.)	~3 hours	~50 min	~70%	

The largest time savings appeared in activities developers often view as overhead: drafting documentation, designing test cases, and preparing pull requests. These tasks, while vital for delivery quality and team alignment, usually offer little intrinsic satisfaction and can interrupt the development flow. By automating the first draft of these artefacts, the AI eased the mental effort of starting from a blank page and allowed developers to focus their energy on refining rather than creating.

Additional observations came up during the evaluation. Developers mentioned that the AI not only sped up individual tasks but also improved overall readiness by revealing assumptions and edge cases earlier in the process. This meant that later development work moved forward with fewer interruptions and less back-and-forth clarification. Reviewers also noted that the greater consistency of AI-generated artefacts made cross-team collaboration smoother since documentation and PR descriptions followed a more predictable structure.

Qualitative feedback backed up these findings. Developers repeatedly said they felt a lighter cognitive load when beginning new tasks, noting that the AI-generated breakdown of requirements helped them understand scope, dependencies, and edge cases faster. Reviewers also emphasised the benefits of standardised AI-generated pull request descriptions, which made it easier to grasp the intent and impact of changes without long clarifications. Importantly, every AI-generated output still required human validation and adjustment. The recorded time savings included this review step, showing that the human-in-the-loop model did not cut into efficiency gains. Instead, it ensured that accuracy, architectural alignment, and compliance standards were maintained. Overall, the results indicate that AI assistance can truly accelerate front-end delivery when used as a structured support tool rather than a replacement for developer judgment.

8. Limitations and Future Potential

While the AI-assisted workflow showed clear productivity and cognitive benefits, several important limitations were noted. A key limitation is the AI's lack of awareness of the airline's internal architecture, coding standards, and system dependencies. Since it operates without access to proprietary repositories or internal documentation, it cannot fully account for established component libraries, API conventions, or architectural patterns. As a result, its outputs often need considerable adaptation to fit internal practices.

The effectiveness of the AI also depends heavily on the quality of the prompts. Vague or poorly structured prompts tend to produce generic or inaccurate responses, while well-crafted ones lead to much more useful outputs. This creates a learning curve and highlights prompt engineering as a growing practical skill (White et al., 2023). There is also a wider organisational risk of over-reliance if governance and validation processes are weak. Developers working under time pressure might accept AI outputs without enough scrutiny, which reinforces the importance of clear policy guidelines.

Furthermore, Generative AI cannot replace developer expertise. Critical thinking, architectural understanding, and domain experience remain vital to assess and adjust AI-generated suggestions. Without human oversight, there is a real risk of subtle errors, false assumptions, or architecturally unsuitable implementations slipping into the codebase unnoticed.

Even with these limitations, the future potential of AI-assisted development in airline environments remains strong. Integration with internal documentation platforms could let AI tools reference approved knowledge when producing outputs. Retrieval-augmented generation (RAG) architectures, which allow AI models to query internal knowledge bases at inference time, represent a particularly promising path for enterprise AI tooling (Lewis et al., 2020). Fine-tuned models trained on company-specific codebases and design systems could eventually turn AI from a general assistant into a highly contextualised and reliable development partner.

9. Reflection on the AI Journey

At the outset, Generative AI was mostly seen as a coding assistant, a tool meant to suggest code snippets or speed up small implementation tasks. However, through hands-on use in real development work, it soon became clear that its real value was different. Instead of replacing the act of coding, Generative AI turned out to be far more useful as a thinking, planning, and structuring partner that supports the cognitive side of software development.

This change in understanding was quite significant. The most time-consuming and mentally demanding parts of front-end development were not about writing code but about interpreting requirements, checking assumptions, planning component structures, and documenting finished work. Generative AI showed strong ability in these areas by turning unstructured information into something actionable, helping to organise thoughts, and greatly reducing the mental load that comes with starting new tasks.

The importance of trust, governance, and human oversight became clear throughout. Early interactions with the AI showed its tendency to make confident yet incorrect assumptions, which reinforced the need for careful prompt design and critical review of all outputs. Over time, a human-in-the-loop approach proved essential, placing AI as a capable but imperfect collaborator rather than a dependable decision-maker.

From a leadership point of view, this work also highlighted that the human side of AI adoption matters just as much as the technical one. Developer worries about job displacement and uneven usage patterns are cultural and organisational issues that call for thoughtful, empathetic leadership instead of purely technical solutions. Building trust in AI tools takes patience and visible commitment from those guiding the change.

In the end, this development reshaped how Generative AI is viewed, moving it from an experimental novelty to a practical and valuable workflow aid. When used with proper governance, clear understanding of its limits, and strong leadership backing, AI becomes a meaningful addition to developer effectiveness, not a replacement for human skill but a powerful extension of it.

Conclusion

This project set out to see whether Generative AI could genuinely improve front-end development efficiency within a regulated airline environment, and the findings suggest that it can, as long as it is introduced with care, structure, and solid governance. By treating AI not as an autonomous coding engine but as a cognitive assistant, the workflow developed in this study managed to reduce the time and mental effort involved in requirement interpretation, test case creation, documentation drafting, and pull request preparation. The measured results, showing average time savings of about 70%, point to the clear operational benefits that AI can bring when applied to the often overlooked yet essential pre- and post-development activities that shape software delivery.

At the same time, the project showed that these gains do not appear automatically. Hallucinations, regulatory limits, and the danger of misplaced trust highlight the importance of human-in-the-loop validation and clear organisational policies. Leadership proved to be a decisive factor: without guidance, reassurance, and structured training, AI adoption risks becoming fragmented, misunderstood, or resisted. When leaders present AI as an augmentation tool, invest in literacy, and embed it into existing workflows, teams are much more likely to adopt it safely and effectively.

In the end, this work demonstrates that Generative AI can be a strong enabler of developer productivity and consistency, but only when combined with thoughtful governance, responsible use, and a culture that values both innovation and oversight. The workflow developed here stands as a practical first step toward integrating AI into everyday development practice, and it sets the groundwork for future improvements such as RAG-based internal knowledge integration or fine-tuned enterprise models. With ongoing leadership support and careful expansion, AI-assisted development could become a core capability within airline software teams, strengthening both delivery performance and the developer experience.

Bibliography

Accenture (2023) A new era of generative AI for everyone. Available at: https://www.accenture.com/us-en/insights/technology/generative-ai

Amershi, S., Chickering, M., Drucker, S.M., Lee, B., Simard, P. and Suh, J. (2019) 'Software engineering for machine learning: A case study', Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 291–300.

Bender, E.M., Gebru, T., McMillan-Major, A. and Shmitchell, S. (2021) 'On the dangers of stochastic parrots: Can language models be too big?', Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp. 610–623.

Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S. and Liang, P. (2021) On the opportunities and risks of foundation models. Stanford University Center for Research on Foundation Models. Available at: https://arxiv.org/abs/2108.07258

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P. and Amodei, D. (2020) 'Language models are few-shot learners', Advances in Neural Information Processing Systems, 33, pp. 1877–1901.

Daugherty, P.R. and Wilson, H.J. (2018) Human + machine: Reimagining work in the age of AI. Boston, MA: Harvard Business Review Press.

European Commission (2021) Proposal for a regulation of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) and amending certain Union legislative acts. COM(2021) 206 final. Brussels: European Commission.

European Union (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88. Available at: https://eur-lex.europa.eu/eli/reg/2016/679/oj

Data Protection Commission (DPC) (2023) Artificial Intelligence and Data Protection: Guidance for Organisations. Dublin: Data Protection Commission. Available at: https://www.dataprotection.ie/en/dpc-guidance/artificial-intelligence

Kotter, J.P. (1996) Leading change. Boston, MA: Harvard Business School Press.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N. and Kiela, D. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', Advances in Neural Information Processing Systems, 33, pp. 9459–9474.

McKinsey & Company (2023) The economic potential of generative AI: The next productivity frontier. McKinsey Global Institute. Available at: https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai

Russell, S. and Norvig, P. (2021) Artificial intelligence: A modern approach. 4th edn. Hoboken, NJ: Pearson Education.

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H. and Schmidt, D.C. (2023) 'A prompt pattern catalog to enhance prompt engineering with ChatGPT', arXiv preprint arXiv:2302.11382. Available at: https://arxiv.org/abs/2302.11382

World Economic Forum (2023) Future of jobs report 2023. Geneva: World Economic Forum. Available at: https://www.weforum.org/reports/the-future-of-jobs-report-2023

Appendix A: AI Usage Guideline for Front-End Developers

Version 1. Internal Draft | Classification: Internal Only

Purpose

This guideline sets out clear principles and limits for how front-end developers may use Generative AI tools within the organisation. It supports practical AI assistance while protecting passenger data, proprietary code, and the organisation's regulatory standing.

Approved Uses of AI Tools

The following activities are approved for AI assistance, provided no restricted information is included in prompts:

- Interpreting and breaking down feature requirements from Jira tickets or briefs
- Generating UI component scaffolds, boilerplate code, and structural templates
- Drafting functional and edge case test scenarios from anonymised requirement descriptions
- Producing first-draft technical documentation and Confluence page summaries
- Generating pull request descriptions based on anonymised change summaries
- Exploring accessibility considerations and best practice guidance
- Prompting for clarifying questions to raise with stakeholders

Restricted Information – Must Not Be Entered Into AI Tools

The following categories of information must never be included in AI prompts or shared with any external AI service:

- Passenger PII: names, email addresses, booking references, payment details, passport information, loyalty account data
- Live or production source code, database schemas, or internal API specifications
- Internal system architecture diagrams, infrastructure configurations, or security details
- Commercially sensitive business logic, pricing algorithms, or revenue management rules
- Credentials, tokens, API keys, or environment configuration files
- Any data classified as Confidential or Restricted under the organisation's data classification policy

Prompting Best Practices

- Always use generic, anonymised, or fictionalised examples in prompts
- Describe requirements in abstract terms rather than sharing actual system details
- Explicitly instruct the AI to state its assumptions and flag areas of uncertainty
- Specify the format and structure of the output required
- Treat all AI outputs as first drafts requiring validation, not final deliverables

Human Validation Requirements

All AI-generated outputs must be reviewed and validated by the developer before use:

- Code: Review for correctness, alignment with internal standards, and absence of hallucinated APIs or dependencies
- Test cases: Verify completeness against acceptance criteria and supplement with domain knowledge
- Documentation: Confirm accuracy, remove AI-generated assumptions presented as fact, and align with internal terminology
- Pull request descriptions: Confirm the description accurately reflects the actual changes made

Approved AI Tools

Only AI tools reviewed and approved by the Technology Leadership and Information Security teams may be used in development workflows. Developers should not use personal AI accounts or unapproved browser extensions for any work-related prompting.

Accountability

The developer who uses an AI tool remains fully responsible for the quality, accuracy, and compliance of the final output. AI assistance does not reduce or transfer individual accountability. Any concerns about data exposure or inappropriate outputs must be reported to the Engineering Lead and Data Protection Officer immediately.

Review and Updates

This guideline will be reviewed quarterly or after any major change in AI tooling, regulatory guidance, or organisational policy. All developers must confirm awareness of the current version during onboarding and annual compliance training.